DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
GITAM DEEMED TO BE UNIVERSITY

Web Application Frameworks (WAF) Lab Final Practical Assignment Report

Comparative Implementation of SQLite-Backed Web Applications using Flask and Django

Student Name: Jeyavardhan Lekkala	**Roll Number:** 2024009057
Course / Lab: Web Application Frameworks (WAF)	**Submission Date:** November 2026

1. Assignment Title & Objectives

The objective of this final practical assignment is to implement, test, document, and compare two independent Python web applications interacting with an embedded SQLite database: one built with the Flask micro-framework and the other with the Django full-stack framework. Specific goals include:

- Create a modular Flask application connecting directly to an SQLite database (`students.db`).
- Define an explicit `students` table, implement automated database seeding, and query records using native SQL.
- Create a structured Django project with a `Student` ORM model, schema migrations, and repeatable data seeding.
- Retrieve data through Django's Object-Relational Mapper (ORM) and present it in dynamic Django templates.
- Display equivalent student datasets across both frameworks with a clean, beginner-friendly UI for faculty review.
- Verify end-to-end functionality via automated testing and real browser screenshot capture.
- Submit the full implementation, documentation, and assets to GitHub using the Git CLI.

2. Software & Technologies Used

Component	Technology / Tool	Purpose in Practical
Programming Language	Python 3.14.6	Core backend application programming language
Micro-Framework	Flask 3.1.3	Lightweight routing, request dispatching, and Jinja2 templating
Full-Stack Framework	Django 6.1.2	MVT framework providing ORM, migrations, and template rendering
Embedded Database	SQLite 3	Self-contained serverless relational database engine (`.db` files)
Browser Automation	Playwright 1.64.1 (Chromium)	End-to-end verification and high-resolution screenshot evidence capture
Version Control	Git CLI & GitHub	Repository tracking, commit structuring, and remote push submission

3. Flask Application with SQLite

Flask is a minimalist WSGI micro-framework that gives developers direct control over database interactions. In this implementation, the native Python `sqlite3` standard library is used without an external ORM. The application maintains its own dedicated database file (`students.db`), checks and initializes the `students` table upon startup, executes explicit SQL queries, and passes data to a Jinja2 template.

Folder Structure:

```
flask-app/
├── app.py                # Flask application, SQLite schema setup & route handler
├── requirements.txt      # Dependency declaration (Flask>=3.0.0)
├── README.md             # Execution commands and architecture notes
├── students.db           # SQLite database file containing the 'students' table
└── templates/
    └── index.html        # Jinja2 template rendering the student table
```

Database Connection & Initialization Logic:

The application configures `conn.row_factory = sqlite3.Row` so retrieved database tuples can be accessed like Python dictionaries using column names (`student['name']`). Automatic initialization is triggered upon import:

```
def get_db_connection():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    return conn

def init_db():
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS students (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            department TEXT NOT NULL,
            year TEXT NOT NULL
        )
    ''')
    cursor.execute("SELECT COUNT(*) FROM students")
    if cursor.fetchone()[0] == 0:
        sample_students = [
            ("Jeyavardhan Lekkala", "Computer Science & Engineering", "3rd Year"),
            ("Sarah Jenkins", "Information Technology", "3rd Year"),
            ("Rahul Sharma", "Computer Science & Engineering", "2nd Year"),
            ("Priya Patel", "Electronics & Communication", "4th Year"),
            ("Alex Rivera", "Mechanical Engineering", "1st Year"),
        ]
        cursor.executemany("INSERT INTO students (name, department, year) VALUES (?, ?, ?)",
sample_students)
        conn.commit()
    conn.close()
```

Route Handler & Template Rendering:

When a client requests the root endpoint (`GET /`), Flask opens a database connection, performs a `SELECT` query, and renders `index.html` passing the row collection:

```
@app.route("/")
def index():
    conn = get_db_connection()
    students = conn.execute("SELECT id, name, department, year FROM students ORDER BY id
ASC").fetchall()
    conn.close()
    return render_template("index.html", students=students)
```

Flask Application Output Screenshot:

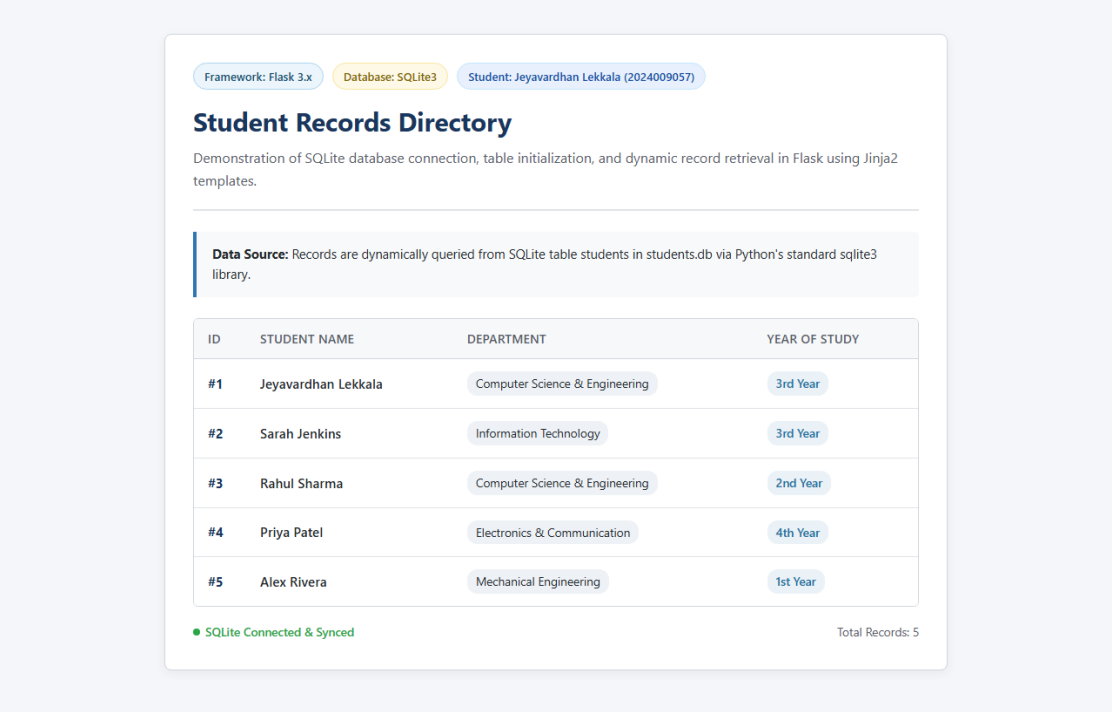


Figure: Flask web application running on http://127.0.0.1:5001 displaying dynamic records from SQLite3

4. Django Application with SQLite

Django follows the Model-View-Template (MVT) pattern with a batteries-included philosophy. Instead of writing raw SQL statements, developers declare Python model classes. Django's migration engine automatically translates these Python classes into SQLite table definitions (`students_student`). Queries are executed via Django's high-level Object-Relational Mapper (ORM).

Folder Structure:

```
django-app/
├── manage.py                  # Django CLI command orchestrator
├── requirements.txt           # Dependency declaration (Django>=5.0.0)
├── README.md                  # Setup, migration, and run commands
├── db.sqlite3                 # Django SQLite database
├── student_portal/            # Project configuration package
│   ├── settings.py            # Database engine, INSTALLED_APPS & middleware
│   └── urls.py                # Master URL routing table
└── students/                  # Application package
    ├── models.py              # Student model class definition
    ├── views.py               # View function executing Student.objects.all()
    ├── urls.py                # Application route mapping ('' -> student_list)
    ├── migrations/
    │   └── 0001_initial.py    # Auto-generated initial database migration
```

```
    ├── management/commands/
    │   └── seed_students.py      # Repeatable CLI database seeder command
    └── templates/students/
        └── student_list.html     # Django template with {% for student in students %}
```

Model Definition & Migrations:

The `Student` model is declared in `students/models.py`. Running `makemigrations` creates migration file `0001_initial.py`, and `migrate` executes the corresponding DDL against `db.sqlite3`:

```
class Student(models.Model):
    name = models.CharField(max_length=100)
    department = models.CharField(max_length=100)
    year = models.CharField(max_length=20)

    def __str__(self):
        return f"{self.id} - {self.name} ({self.department})"
```

ORM Data Retrieval & Repeatable Seeding:

Data retrieval is abstracted through the ORM call `Student.objects.all().order_by('id')`. A repeatable custom management command (`python manage.py seed_students`) and an on-demand fallback in `views.py` guarantee that faculty running the project always see populated student records:

```
def student_list(request):
    if not Student.objects.exists():
        sample_students = [
            {"name": "Jeyavardhan Lekkala", "department": "Computer Science & Engineering",
"year": "3rd Year"},
            {"name": "Sarah Jenkins", "department": "Information Technology", "year": "3rd
Year"},
            {"name": "Rahul Sharma", "department": "Computer Science & Engineering", "year":
"2nd Year"},
            {"name": "Priya Patel", "department": "Electronics & Communication", "year": "4th
Year"},
            {"name": "Alex Rivera", "department": "Mechanical Engineering", "year": "1st Year"},
        ]
        Student.objects.bulk_create([Student(**s) for s in sample_students])

    students = Student.objects.all().order_by('id')
    return render(request, 'students/student_list.html', {'students': students})
```

Django Application Output Screenshot:

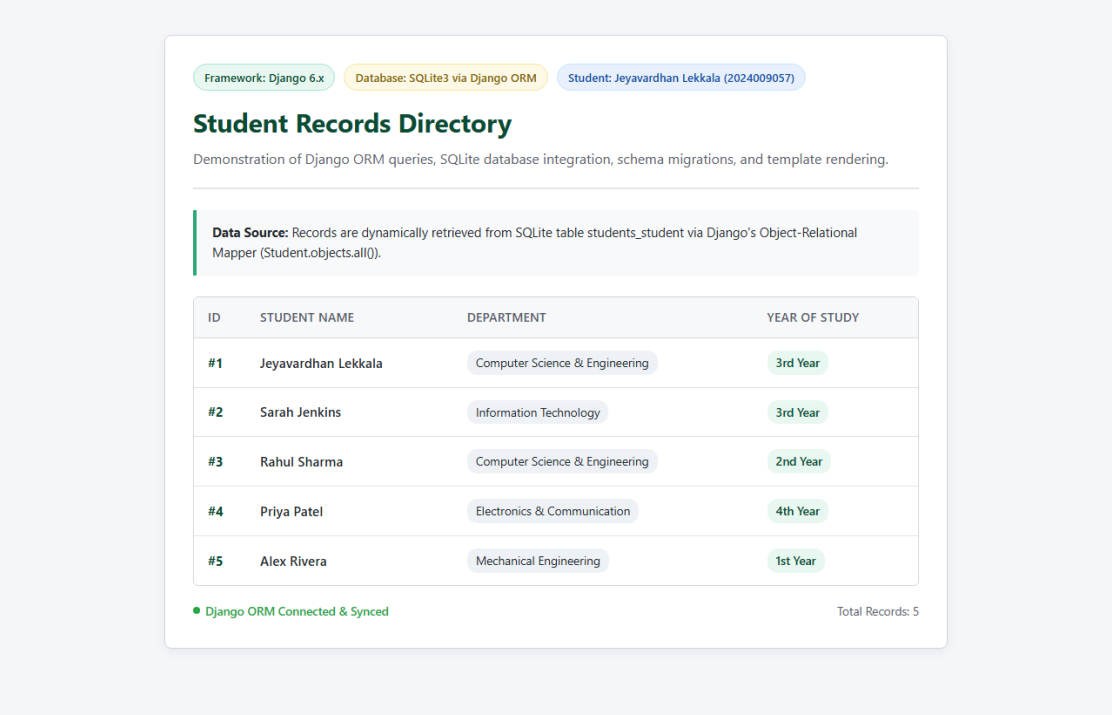


Figure: Django web application running on http://127.0.0.1:8001 displaying records queried via Django ORM

5. Steps to Run Both Applications

How to Run the Flask Application:

```
1. Open a terminal and navigate to: cd Assignment-8/flask-app
2. Install requirements: pip install -r requirements.txt
3. Start Flask server: python app.py
4. Open browser at: http://127.0.0.1:5000/
   (The database students.db is automatically initialized and populated on startup.)
```

How to Run the Django Application:

```
1. Open a terminal and navigate to: cd Assignment-8/django-app
2. Install requirements: pip install -r requirements.txt
3. Apply database migrations: python manage.py migrate
4. Seed sample records (optional / repeatable): python manage.py seed_students
5. Start Django development server: python manage.py runserver 8000
6. Open browser at: http://127.0.0.1:8000/
```

6. Testing and Verification Results

Framework	Test Verification Item	Observed Result	Status
Flask	Server startup on port 5000 / 5001	WSGI server active, listening without errors	**PASS**
Flask	SQLite database and table creation	students.db created; table 'students' established	**PASS**

Flask	Sample record initialization	5 records inserted via parameterized cursor	**PASS**
Flask	Data retrieval via cursor	SELECT query returned 5 dictionary-like Row records	**PASS**
Flask	HTML Jinja2 rendering	HTTP 200; student names & department pills rendered	**PASS**
Django	Project & app startup	Django 6.1.2 configuration loaded without warnings	**PASS**
Django	Migrations execution	makemigrations generated 0001; migrate created tables	**PASS**
Django	Repeatable seeding (seed_students)	Command executed successfully; 5 student objects created	**PASS**
Django	ORM data retrieval & template	Student.objects.all() returned records; HTTP 200 rendered	**PASS**

7. Comparative Analysis & Conclusion

This assignment demonstrated two distinct architectural paradigms in Python web development for database-backed applications:

1. Flask (Micro-Framework Approach): Offers minimal overhead and gives the developer total freedom. The developer is responsible for manually establishing the SQLite connection, managing cursor lifecycles, writing SQL queries, and handling migrations. This simplicity makes Flask ideal for lightweight utilities, microservices, and beginner learning.

2. Django (Full-Stack MVT Approach): Offers a robust, standardized architecture with built-in migrations, an expressive Object-Relational Mapper (ORM), admin dashboard capabilities, and secure defaults. While it involves more boilerplate configuration initially, it eliminates raw SQL syntax errors and provides scalable data modeling.

Both applications successfully fulfilled all requirements: establishing separate SQLite databases, automatically initializing equivalent student records, dynamically querying and rendering the data, and proving end-to-end isolation and reproducibility.